CMS Team Project Interactive Visual Computing

Recipe Emotional Advisor

August 12, 2026

Technische Universität Dresden

The Interactive Media Lab Dresden(Chair of Multimedia Technology)
Technische Universität Dresden
CMS Team Project Interactive Visual Computing

Team Project Report

CMS Team Project Interactive Visual Computing

Recipe Emotional Advisor

1. Reviewer Susmita Sunil Khadse
The Interactive Media Lab Dresden(Chair of Multimedia Technology)
Technische Universität Dresden

2. Reviewer Julián Méndez
The Interactive Media Lab Dresden(Chair of Multimedia Technology)
Technische Universität Dresden

Supervisor Prof. Dr.-Ing. Raimund Dachselt

Team Members

Natasha Hrycan	5258443
Shuyi Yu	5266721
Atul Kumar Pandey	5263456

Recipe Emotional Advisor
CMS Team Project Interactive Visual Computing
Team Project Report, August 12, 2026
Reviewers: Susmita Sunil Khadse Supervisors: Prof. Dr.-Ing. Raimund Dachselt

Technische Universität Dresden
CMS Team Project Interactive Visual Computing
Technische Universität Dresden
The Interactive Media Lab Dresden(Chair of Multimedia Technology)
Helmholtzstr. 10
01069 and Dresden

1 Introduction

This team project works on building up a recipe LLM data storytelling platform, which hopefully goes beyond a conventional recipe recommendation.

The project not only followed a typical way of recipe recommender to collect user preference and give back suitable recipes, but also applying interactive process and data storytelling to help this recommendation more understandable and explicit for users.

The system therefore combines several types of information: user preferences, recipe metadata, nutrition values, human written story and LLM generated narrative. Successfully integrating all above information into one complete journey, the system starts with helping users building up their profiles through front end question interactions, then backend uses the structured answers to find the most suitable recipe along with the matching stories and send back to front end for users to read.

The project could be divided into three main parts, front end part for user interaction and visual design with presentation, backend part for processing use data and outputing data fields, LLM part for generating data storytelling based on human training.

Experimental Framework

2

2.1 Front-end Development

Focusing on the main goal, the front end has to collect user preferences. However, instead of using a standard form, we found out that making a game-like interactive kitchen could make this platform more interesting. Thus, we built a kitchen setup related to the recipe topic we chose, and added separate interactions to each part of the kitchen to replace basic texts. The front end also needed to figure out the layout of the final modal for all the information we wanted to show to users. Owing to this, we decided to put the two stories from human and LLM above all and in parallel, as they are the most important outcomes of our project. At the same time, we kept the recipe description and recommendation reasons in different sections, instead of mixing them with the stories. This makes the final modal clearer, because the human story, LLM story, basic recipe information and recommendation explanation all have their own place.

To realize the ideas above, the front end was implemented with Svelte and Vite. The interface was divided into several parts for better organization: the kitchen stage, the question panel, the sidebar and profile panel, and the recipe story modal. In the interaction design, each kitchen object represents one user preference, for example, the avatar for user type and the recycle bin for allergies. Each object would be highlighted when hovered over or when answering the question it stands for. In this way, users are not simply filling in a form, but building their recipe profile step by step through the kitchen scene.

Another important part of front-end design was to find out the way to connect the front end and backend. After collecting the user answers, the front end converts them into a structured payload. For example, user type would be mapped to persona, cooking time mapped to time, selected ingredients mapped to ingredient groups, and allergies mapped to avoid rules. This step ensures that the visual interaction can still produce clean and consistent data for the whole system. Other than this, the front end also created an adapter for the backend response. This was done because during the development, the backend response format changed several times, so it would be wiser for the front end to use an adapter to normalize the returned data before passing it to the modal, instead of letting the visual components depend directly on raw backend fields. This also helped us when the human story field was confirmed later. After the backend added human story, the front end only needed to read this field through the adapter and then show it on the left side of the modal. In this way, the modal could compare the human-written story and LLM-generated story more directly.

During the development, there were several challenges. To start with, when trying to make the experience more engaging, the highlights did not work right. The highlights looked unnatural with rectangular hitboxes. In order to solve this, we separated different layers of the kitchen background, so that only the selected item would be highlighted. Another challenge was to support data storytelling visually. At first, the final modal was still mainly based on plain text, which was not enough for showing the data storytelling idea. After discussion, a nutrition snapshot using donut charts was added to give more explicit feedback. Calories are shown based on a 2000 kcal reference diet, while other nutrition values use the percentage daily value fields from the backend. Time, steps and difficulty are still shown as texts because they are easier to read directly.

Another practical challenge was testing the final result page when the backend was not fully ready. To solve this, we used mock recipe data first, so the recipe modal, nutrition charts and story sections could be tested separately. After these parts worked, the mock data was removed and the real backend request was connected again. This made the development process more stable, because the front-end layout and interaction could be checked before the full system was ready.

In conclusion, the final front end brings the main parts of the project together in a working interface. Users can build their profile through the kitchen stage, send their choices to the backend, receive a recipe recommendation, and view the result through story comparison and nutrition charts. Compared with the mid-term version, it is no longer just a visual prototype. It now connects interaction, backend data, human and LLM stories, and visual explanation into one complete user flow. Therefore, front end part of the project was not only about building the visual page, but also about making sure that user interaction, backend data and storytelling results could work together smoothly.

2.2 Back-end Development

The backend of this Emotional Recipe Advisor is connecting the user front-end to the dataset and the LLM Model components. It receives the user's choices/selection from the wizard interface and then it filters out the most suitable recipe from the Food.com dataset. After this it assembles the structured response that the front end displays as the recipe card and the LLM Model uses it for Story generation. The Backend service is implemented using Fast API which we have chosen because it is modern Python web framework which provides automatic request validation through the models connected to it. The service has a primary endpoint at Api of recommended-recipe which accepts all fields required for both the front-end display and the LLM story generation step.

As we wanted to separate recipe selection from narrative generation we made a central design for this application. The backend is totally responsible for choosing the recipe by performing filtering on Real data so that it should not generate any narrative text by itself. This separation idea is taken from the Aletheia system [Fu+24] this shows that the generated claims in data storytelling should always be grounded in verified data. As all of the factual contents like ingredient lists,cooking times, nutritional values and dietary suitability are always drawn from the dataset and never generated by a LLM model so this system guarantees that the story that is written by LLM is built on a factually accurate foundation.The fields 'tagline' and 'Human-story' are returned by the back end, which the LLM section describes as the primary inputs for generating structured generated narrative and it produces it deterministically from the recipe's dataset attributes and the user's chosen option and not from any LLM model.

Nutritional goal tokens are mapped to one of the four internals ranking strategies which are muscle,lose weight, eat healthier or to save money.Time ranges are converted to integer upper bounds. Ingredient category labels are expanded as lists of representative ingredient keywords drawn from the dataset's ingredient vocabulary, so that a preference for `grains-starches` generates matches against rice, pasta, quinoa, potato, oats, and similar terms. Allergen tokens are expanded through a synonym map that ensures for example that a request to avoid `dairy` also removes recipes containing butter, cream, yogurt, and whey. The translation layer is deliberately forgiving as values it does not recognise fall back to sensible defaults rather than raising errors, which keeps the system robust against unexpected front-end values without requiring changes to both codebases simultaneously.

The dataset used in the system is the Food.com Recipes corpus by Majumder et al. [Maj+19], which contains 231,637 recipes and each record has a name, unique identifier, ingredient list, step-by-step instructions, descriptive tags, and a nutrition tuple. The nutrition tuple encodes seven values in a fixed order that is absolute calories followed by six nutrients expressed as percentages of recommended daily intake, specifically total fat, sugar, sodium, protein, saturated fat, and carbohydrates. The back end labels all nutrition fields explicitly with the `pct-dv` throughout the response object and the LLM prompt is extended with an explicit clarification that fields ending in this suffix represent percentages of daily value rather than absolute quantities and this directly addresses the hallucination failures observed during the story generation experiments described in the LLM section.

Before the dataset is used for filtering, it undergoes a cleaning and preprocessing stage that runs once at server startup and caches the result in memory. Recipes with less than four or more than twenty ingredients are removed as these are the recipes with less than three preparation steps or cooking times outside the range of five to two hundred and forty minutes, since values outside this window are typically data entry errors rather than genuine recipe characteristics. A nutrition sanity filter removes recipes whose per-serving values exceed desired thresholds that is specifically more than 1,500 calories or more than 120 percent daily protein or more than 200 percent daily sugar or more than 200 percent daily sodium or more than 150 percent daily carbohydrates. These outliers arise almost exclusively from bulk-batch recipes whose nutrition figures describe the yield of an entire batch rather than a single serving. Without this filter the protein-ranking logic consistently gives such recipes as top results such as a bulk pizza dough recipe recording 463 percent daily protein would rank far above any genuine high-protein meal. Finally recipes identified as cooking components rather than complete dishes are not considered including those whose names or tags classify them as doughs, sauces, marinades, dressings, or glazes. After all cleaning steps, approximately 27,123 recipes remain available for recommendation.

The filtering and ranking function checks to identify recipes that match the user's profile. The dish type filter retains only recipes whose tag list contains at least one of the tags associated with the requested meal category. The time filter retains only recipes whose recorded cooking time does not exceed the user's stated maximum. The ingredient preference filter gives the recipes whose ingredient list contains at least one of the expanded keywords for the user's selected category and using any-match rather than all-match logic because requiring every keyword to appear simultaneously would eliminate almost every candidate. The allergen removal filter removes recipes containing any of the expanded synonym keywords for the given allergens that ensures dietary safety deterministically on the data rather than relying on the language model. A step-count taken from the user's effort level is applied with beginner effort mapping to a maximum of six steps otherwise intermediate to twelve. Among the recipes that passes all active filters the candidates that are ranked by the user's nutrition goal are descending protein content for the build-muscle goal as the increasing calorie count for the weight-loss goal and increasing ingredient count as a substitution for simplicity and also to save the money and unmodified filtered order for the eating healthier goal.

Once a recipe has been selected then the back end sends the response object that is returned to the front end and is used as input to the language model. The response includes the recipe identifier, name, description, tags, cooking time, and step count taken directly from the dataset as well as the full nutrition tuple restructured into a labelled object with different field names. The `tagline` field is generated deterministically from the user's persona, dish type preference and the selected recipe's cooking time. The `Humanstory` field serve as a baseline for emotional quality and a control sample against which LLM-generated stories can be compared. These two fields serve as the primary structured input to the language model which are described in the LLM Story Generation section that is providing a factually accurate and concise summary of the recommendation story that the model is then instructed to make it into an emotionally engaging narrative paragraph.

2.3 LLM Story Generation

Taking into account that the main goal of this project was to create an interactive platform where we could show and compare human and LLM generated stories in a recipe advisor format, we decided to review similar works on this area. The answer of the model should follow a logical order, ideally presenting an engaging opening sentence that highlights empathy with the user, followed by factual information related to the recipe, and ending with another emotional motivational sentence. We will have to pay attention to the emotional coherence of the answers, making sure the model focuses on the same persona while generating the output. The findings of this paper were crucial for formatting the prompt used to query results from the LLM

LLMs can show great adaptability in matching a certain tone and this can be further clarified through a rigorous set of rules. For this reason, few-shot prompting and rules are more appropriate approach than training or fine-tuning a model [Sah+24]. The expected output was JSON format answer, domain-specific and with an emotional component to the answer that could be controlled with a small number of examples.

The input was structured following the instruction based format popularized by Stanford Alpaca [Tao+23], with explicit instructions followed by recipe specific input and few shot examples. The Alpaca format provides a clean and easy to reproduce structure, it does not require additional special tokens and can be widely adopted by most state of the art models. On the other hand, the use of # symbols can sometimes cause erratic behavior and since there is no end of sequence token, some models may oversee the rule that limits the amount of words. An example of the prompt used is available in the Appendix section A.1.

We tested initially several Instruct models locally for the generation of the stories: Qwen 2.5 Instruct, Mistral Instruct, and Llama 3.1 Instruct. From these, Qwen 2.5 had he most reasonable outputs, but after consultation with our supervisors and further testing, the final experiments were conducted using GPT-OSS-120B, hosted by ScaDS.AI. Together with the prompt and the few shot examples we intended to show the LLM what is the expected relationship between the input recipe information, user persona, nutritional information, and the output generated story.

The final prompt contained a set of explicit rules decided to constrain the content, length of text and tone of the generated stories. The model was instructed to use only information contained in the input JSON and to treat the `tagline` as a summary of the user's persona and `why_it_fits` as the main explanation for the recipe selection. Each story was required to contain between 150 and 200 words and to form a single rich paragraph. In addition, the prompt specified a predefined narrative structure, beginning with an emotionally aware hook, followed by a description of the recipe, cooking time, number of steps, difficulty, ingredients, and nutritional information, before explaining why the recipe was appropriate for the user and ending with a practical or emotional closing.

After the first rounds of testing, we had to pay particular attention was given to nutritional information. We required calories and protein percentage of daily value whenever available, while other nutritional values could be included when relevant. The prompt explicitly clarified that fields ending in `_pct_dv` represented percentages of daily value rather than grams, in order to reduce incorrect interpretations of the nutritional data. The model was also instructed not to invent ingredients, cooking times, nutritional values, reviews, or health claims, and not to recommend ingredients that were mentioned only in a description but were absent from the ingredient list. Finally, the output was constrained to valid JSON with a fixed structure, with no Markdown or additional text outside the JSON object.

A significant part of the prompt fine tuning process involved addressing inconsistencies in the outputs of the Instruct tuned models. In early experiments, the models frequently failed to respect the requested 150-200 word range. Some generations were considerably longer than requested, while others were extremely short, occasionally consisting of only two or three sentences. This problem was particularly noticeable when multiple recipes were queried simultaneously. Although one generated story might have an appropriate length, another response in the same query could be substantially shorter. The tone was also inconsistent at times, with some outputs adopting the intended emotional and personal style while others remained largely neutral. Towards the end of the process, the use of emojis was also experimentally introduced. In some answers the LLM used emojis in a natural manner, while in other responses it had completely forgotten and did not use any emojis at all. Another important issue was the handling of nutritional information. The models occasionally altered or hallucinated nutritional values despite these values being explicitly provided in the input. Additional prompt rules were therefore introduced to reinforce the distinction between absolute nutritional quantities and percentage daily values and to require the model to ground nutritional statements directly in the input data.

The prompt was consequently refined iteratively through trial and error. Rules were added or modified whenever a recurring failure pattern was observed, with the goal of improving consistency across all generated stories rather than optimizing a single output. At several stages, additional language models, including ChatGPT, were consulted to suggest prompt formulations or additional constraints that could address specific failures. The final prompt therefore emerged as the result of an iterative prompt engineering process in which generated outputs were inspected, recurring patterns in hallucination or failure were identified and explicit instructions were progressively added to address them.

Results 3

After successfully implementing the pipeline for a demo, we conducted a small user case study. The main goal of this survey was to understand how do users react with the stories that we present them. In the following sections we will be explaining in more detail how we conducted this study and discuss the results, which lead to future work suggestions

3.1 User Case Study

In order to test in an unbiased environment how emotional and human were the responses generated by the LLM, we conducted a user case study with 8 people. The users were explained that the goal of our project is to generate an interactive interface where users can answer a set of questions and filters, so that they can receive a suggestion of a recipe that fits their requirements. Besides a recipe suggestion, the interface also provides two stories introducing the recipe suggestion and nutritional information displayed in a visual representation. After the explanation, users were shown 3 examples of cases to judge. We blurred the titles "Human generated story" and "LLM generated story" from our original demo, because we wanted to ask users which of the stories they thought was written by a human, and which one by an LLM. Additionally, we also asked the users to rate the emotional tone of the stories from 1 to 5, 5 being a very emotionally engaging story and 1 being a story with a rather neutral or emotionless tone.

The most interesting metric for the study was the guess of the users about which of the stories was human and LLM generated. This is a relevant point considering that LLMs have improved considerably in recent times on the quality of the text that it is generating, sometimes even making it difficult for GenAI text checking programs to differentiate effectively. It is important to note, however, that the displayed LLM generated stories underwent fine-tuning in the prompt and several attempts before being selected as the final version.

The full survey collection of our user case study is collected in Appendix A.1, and the summarized results are presented in Table 3.1. Besides the feedback received from the users, we also manually inspected the LLM generated stories for the accuracy of numerical nutritional information displayed in the story. We considered this to be a relevant factor as we are concerned with data storytelling, where numerical factual data should be represented accurately and cannot leave room to hallucinations of the LLM. This verification was therefore performed before the user study, allowing the study itself to focus primarily on the perceived storytelling quality, emotional tone, and users' perception of whether the content was human or LLM generated.

Example	Average emotional score	Flagged as LLM-generated
Example 1	2.13	87.5%
Example 2	2.50	87.5%
Example 3	2.75	87.5%

Tab. 3.1.: Summary of the user study results for the three recipe storytelling examples. Emotional scores range from 1 to 5, with higher values indicating a stronger perceived emotional quality.

As shown in Table 3.1, the average emotional score increased across the three examples, from 2.13 for Example 1 to 2.50 for Example 2 and 2.75 for Example 3. However, this increase was not accompanied by a change in the perceived origin of the texts: 87.5% of participants classified each of the three examples as LLM generated. The individual comments provide some indication of the characteristics participants used to make this judgement, including repetitive structures, explicit phrasing, short sentences, and stylistic conventions associated with LLM-generated text. After completing the survey, further discussion with the participants about storytelling and the characteristics of LLM-generated content also indicated that most participants perceived the presented answers as LLM-generated.

These observations should be interpreted cautiously given the small number of participants and the exploratory nature of the study. Nevertheless, they suggest that users may have developed increasingly specific expectations about the characteristics of LLM-generated text. In particular, some participants identified relatively natural or "life-like" writing as LLM-generated, suggesting that perceived authorship and perceived naturalness are not necessarily directly related. One possible interpretation is that the distinction between human- and LLM-generated writing is becoming increasingly difficult to establish based solely on writing style. As LLM-generated text becomes more natural and human writers potentially become more familiar with stylistic patterns commonly associated with LLMs, the textual characteristics used to distinguish between the two may increasingly overlap. This interpretation remains speculative and would require a larger and more controlled study to be investigated systematically.

4 Conclusion

This Emotional Recipe Advisor project demonstrates how a carefully engineered combination of structured data, deterministic backend logic, and controlled LLM storytelling can produce a safe, reliable, and emotionally engaging user experience. This system integrates three major components into a seamless pipeline that are a Svelte-based frontend wizard that collects user preferences, a FastAPI backend engine that filters and ranks recipes using real Food.com data, and a LLM storytelling module accessed through the Scads interface to generate warm and personalized narratives.

The user study with eight participants provides meaningful insight into the system's narrative performance. Emotional scores ranged from 2.13 to 2.75, indicating moderate engagement, while 87.5% of stories were flagged as LLM-generated. Participants frequently gave feedback of repetitive structure and stylistic patterns typical of LLMs. These results show that the system produces consistent and safe narratives but also highlight the challenge of achieving human-like emotional richness. As LLMs continue to evolve, this distinguishing human and machine-generated text may become increasingly difficult and underscoring the importance of controlled evaluation.Overall, the project successfully delivers a complete end-to-end AI product which is technically sound, user-tested, safe by design and ready for future expansion.

4.1 Future Work

Several promising directions can enhance the system's emotional quality, personalization, and robustness:

- **A**dd an interactive portion-size slider: A valuable extension to the frontend would be an interactive portion-size slider. Users could adjust serving sizes dynamically and the backend would recalculate ingredient quantities and nutrition values in real time. This would improve usability, personalization, and practicality for everyday cooking.
- **I**ntegrate real cost estimation: A more accurate approach could pull real-time ingredient prices, estimate total recipe cost, and rank recipes by affordability.

Generative AI Disclosure

AI was used as part of the development of this project, since the goal was to evaluate LLM responses for data story generation. LLMs were used as an assistance tool for literature research, brainstorming, code assistance and refining the prompts used for querying results for the project.

Bibliography

[Fu+24] Yu Fu, Shunan Guo, Jane Hoffswell, et al. “"The Data Says Otherwise" — Towards Automated Fact-checking and Communication of Data Claims”. In: *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*. UIST ’24. Pittsburgh, PA, USA: Association for Computing Machinery, 2024 (cit. on p. 5).

[Maj+19] Bodhisattwa Prasad Majumder, Shuyang Li, Jianmo Ni, and Julian McAuley. “Generating Personalized Recipes from Historical User Preferences”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong, China: Association for Computational Linguistics, 2019, pp. 5976–5982 (cit. on p. 5).

[Sah+24] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, et al. “A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications”. In: *arXiv preprint arXiv:2402.07927* (2024) (cit. on p. 7).

[Tao+23] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, et al. “Alpaca: A Strong, Replicable Instruction-Following Model”. In: *Stanford Center for Research on Foundation Models* (2023) (cit. on p. 7).

Appendix

A

A.1 Final LLM Prompt

The final prompt used for generating the LLM-based recipe stories is provided in Listing A.1. The prompt was designed to constrain the generated stories to the information provided in the input JSON while enforcing a predefined narrative structure and consistent use of nutritional information.

```
Instruction

You are a recipe storytelling assistant.
Write a short emotional recipe recommendation using the given recipe
and the persona implied by tagline and why_it_fits.

Rules:

Use only facts from the input JSON.

Treat tagline as the user/persona summary.

Treat why_it_fits as the main explanation for why this recipe was selected.

Write 150 to 200 words.

The response must be one rich paragraph, not a short comment.

The story must follow this structure:

    Start with an emotionally aware hook for the persona.

    Name the recipe and describe what it actually is.

    Mention cooking time, n_steps, and difficulty.

    Mention at least 2 ingredients.

    Emphasize nutrition data using at least 4 numbers when available.
```

```

    Explain why the recipe fits the user.

    End with a practical or emotional closing sentence.

Always mention calories when available.

Always mention protein_pct_dv when available.

Mention carbohydrates_pct_dv, total_fat_pct_dv, sodium_pct_dv,
sugar_pct_dv, or saturated_fat_pct_dv when useful.

Nutrition fields ending in _pct_dv are percentages of daily value,
not grams. Never call them grams.

Use phrases like "117% of the daily protein value", not
"117 grams of protein".

If nutrition values are very high, describe the recipe as rich,
filling, satisfying, indulgent, salty, fatty, or calorie-dense
rather than light or healthy.

If a nutrition value is low, you may describe it as low only when
the number supports it.

Use warm, vivid language, but keep the story grounded in data.

Use 1 to 3 emojis if they naturally fit the emotional tone.

Do not overuse emojis.

Do not invent ingredients, cooking times, nutrition values, reviews,
or health claims.

Do not recommend optional ingredients from the description unless
they also appear in ingredients.

Output valid JSON only.

Do not write markdown.

Do not add text outside the JSON.

The output JSON must have exactly this structure:
```

```
{
    "llm_story": "150 to 200 word emotional recipe story with recipe
                  description, cooking difficulty, and nutrition data"
}
```

Listing A.1: Final prompt used for LLM-based recipe story generation

A.2 User case study results

User	Example 1		Example 2		Example 3		Comments
	Origin	Score	Origin	Score	Origin	Score	
User 1	LLM	1	LLM	1	LLM	1	Left side seems human, but not entirely. Humans tend to explain more, left side has very short sentences, which makes it a bit towards LLM. Whereas right side is entirely following the LLM answer structure.
User 2	LLM	3	LLM	3	LLM	3	No comments.
User 3	LLM	2	LLM	3	LLM	3	It's not looking natural, same pattern is getting repeated.
User 4	LLM	2	LLM	2	LLM	2	No comments.
User 5	Human	2	LLM	3	LLM	5	On 3 can't really tell which one is LLM, I lean towards right due to Oxford comma and stuff like that.
User 6	LLM	1	Human	3	Human	2	First one of the first example sounds very much like ChatGPT.
User 7	LLM	1	LLM	1	LLM	2	All are very explicit which are written by LLM.
User 8	LLM	3	LLM	4	LLM	4	The first one was not that good, but the last two are quite life-like.

Tab. A.1.: Complete responses collected during the user study. Participants indicated whether they perceived each example as LLM-generated or human-written and assigned an emotional score from 1 to 5.

Continued on next page